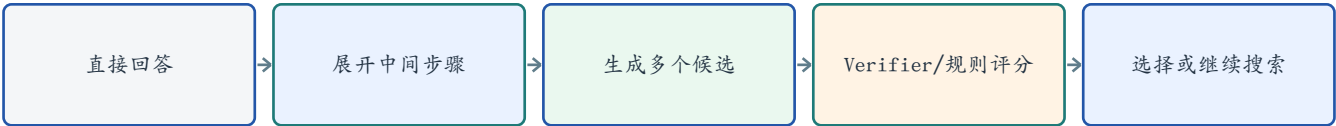


第18讲

Reasoning 与 Test-Time Compute

为什么模型“多想一会儿”会变强？

本讲主线 把“推理”从神秘能力拆成可观察的计算过程：中间步骤、多个候选、验证器、搜索，以及这些机制如何用更多推理阶段计算换取更高成功率。



从单次前向生成到 test-time compute 的基本扩展

学习定位：零基础理解 + 为 LLM / Agent Systems 科研建立实验设计直觉

建议：先看懂概念和例子，再运行配套代码。公式只要求理解含义，不要求手推复杂梯度。

1. 本讲学完，你应该能回答什么？

- Reasoning（推理）在 LLM 语境里通常指什么？它和“模型真的像人一样思考”是不是一回事？
- Chain-of-Thought（CoT）为什么有时能提高复杂任务成功率？
- Test-Time Compute 和训练阶段算力有什么区别？
- Best-of-N、Self-Consistency、Verifier、Search 分别在增加什么计算？
- 为什么“生成更多候选”本身不会自动变强，关键还在于选择与验证？
- 为什么错误相关性会让 test-time scaling 很快遇到收益递减？
- 做 Agent / Multi-Agent 论文时，为什么必须设置 compute-matched baseline？

一句话预告

训练阶段 scaling 是“花更多算力把参数学得更好”；test-time scaling 是“参数基本不变，在解决当前问题时多花一些计算”。

2. 先把“推理”这个词去神秘化

在日常语言里，“推理”很容易让人想到人的意识、内心思考和逻辑证明。但在大语言模型研究中，我们通常先采用一个更可操作的定义：模型为了得到答案，产生、维护或搜索一系列中间计算状态/中间文本步骤。

这些中间状态可以表现为文字形式的推理链，也可以是模型内部隐藏状态、工具调用结果、程序执行、候选解、树搜索节点等。科研时应该优先研究这些可观测、可复现、可操控的对象，而不是直接从输出推断模型具有某种人类式意识。

事实 / 推断 /
不确定性

事实：增加中间计算在许多复杂任务上可以提高性能。推断：中间计算可能帮助分解问题、维护状态、纠错或搜索。仍不确定：自然语言推理轨迹与模型内部真实因果计算之间到底有多一致。

通俗比喻

一个学生做两位数乘法，如果要求“看到题目后 1 秒钟只报最终答案”，容易出错；如果允许他写草稿，把乘法拆成几个小步骤，再检查一次，成功率通常会更高。LLM 的 test-time reasoning 可以先用这个比喻理解：不是立刻换了一个更聪明的大脑，而是给同一个系统更多计算过程。

3. Direct Answer 与 Chain-of-Thought

方式	计算形态	优点	风险
Direct / Short answer	快速从上下文直接生成最终答案	低延迟、低成本；简单问题通常足够	复杂任务可能缺少中间状态，容易跳步
Chain-of-Thought	先生成若干中间步骤，再给结论	可分解、多步维护、便于局部检查	步骤也可能是错的；可能越想越偏；成本更高

例如题目：一家商店先把 200 元商品打 8 折，再在折后价基础上打 9 折。直接回答可能立即生成一个数字；CoT 则可以显式形成：

Step 1: $200 \times 0.8 = 160$

Step 2: $160 \times 0.9 = 144$

Answer: 144

这里中间步骤的价值不在于“文字变长”本身，而在于它为后续 token 提供了新的上下文状态。模型第二步看到的不再只是原题，还能看到第一步得到的 160。

常见误区

CoT 长，不等于 CoT 对。一个错误的第一步也会成为后续生成的新上下文，从而把错误继续放大。

4. 为什么多写中间步骤可能有帮助？

可以把原因拆成四种非常直观的机制。它们未必对每道题同时成立，但足以建立研究直觉。

机制	底层直觉	例子
问题分解	把一个高难映射拆成多个较简单映射	先算子问题 A，再用 A 的结果处理 B
外部工作记忆	已生成 token 进入上下文，后续可以继续读取	把中间数字、变量、计划写出来
增加搜索机会	同一个问题可探索多条候选路径	数学题尝试不同解法、代码尝试不同修复
增加验证机会	有了中间结果，才有机会逐步检查或调用工具	用计算器核验算术，用测试用例核验代码

因此，“多想”真正有价值的条件不是简单增加 token，而是额外 token 改变了后续决策所依据的信息，或者帮助系统探索/筛选更好的候选。

5. Training Compute 与 Test-Time Compute 是两条不同的轴



6. Test-Time Compute 可以花在哪里？

方法	额外计算花在哪里	核心瓶颈
Longer reasoning / CoT	单条轨迹写更多中间步骤	一条轨迹早期出错可能一路放大
Self-Consistency	采样多条推理轨迹，再做投票/聚合	候选是否足够多样；答案是否适合投票
Best-of-N	生成 N 个候选，然后选择最好的	选择器 / verifier 是否可靠
Search	树状扩展、回溯、剪枝、保留有希望的节点	分支爆炸；评估函数质量
Tool / Code execution	调用计算器、Python、检索、测试环境	工具可靠性、接口、权限和成本
Multi-Agent deliberation	让多个 agent 产生/批判/整合候选	额外调用很多；错误相关和群体偏差

这些方法的共同本质可以写成：

More candidate computation + Better evaluation / aggregation -> Potentially better final answer

注意关键词是 `potentially`。如果候选高度相同，或者 `verifier` 本身很差，额外算力可能几乎没有收益。

7. Best-of-N：最简单的“花更多推理算力”

假设模型对一道题生成 N 个不同候选，然后交给一个评分器选择。最理想的情况是：只要 N 个候选里有一个正确，评分器就一定能把它找出来。这个理想评分器可以叫 Oracle Verifier。

如果每次独立采样成功率都是 p ，并且真正独立，那么“至少一次成功”的概率是：

$$P(\text{at least one success}) = 1 - (1 - p)^N$$

例如 $p = 0.6$ ： $N=1$ 时是 0.6； $N=4$ 时理论上“至少有一个成功候选”约为 0.974。听起来非常强，但现实里有两个大问题。

- 错误并不独立：同一个模型在同一道难题上往往会反复犯同一种错。
- 你不知道哪个候选正确：需要 verifier、规则、测试环境或其他可靠信号去识别。

底层结论

Best-of- N 的上限取决于“候选多样性”，实际效果还取决于“选择器质量”。只有 N 本身不是能力。

8. Self-Consistency：为什么“多条路投票”有时有效？

Self-Consistency 的典型思路是：对同一道推理题采样多条 reasoning path，然后根据最终答案做多数投票。直觉上，如果错误路径比较分散，而正确答案比较稳定，多数投票可以提升可靠性。

例如 7 条轨迹最终答案分别是：

144, 144, 144, 160, 144, 180, 160

那么 144 获得 4 票，系统可以把它作为最终答案。

但这里隐含一个很强的假设：错误要有足够的去相关性。如果 7 条轨迹都被同一个错误诱因带偏，投票只会形成“错误共识”。

与 Agent Systems
的连接

多 Agent 讨论和 Self-Consistency 有很深的结构相似性：真正重要的不只是“数量”，而是候选之间的错误相关性，以及聚合机制是否能识别高质量信息。

9. Verifier：真正决定 test-time scaling 能不能兑现

生成候选并不难，难的是判断哪个候选更好。Verifier（验证器/评分器）就是 test-time compute 系统里的关键组件。

Verifier 类型	依据	优点	局限
规则 / 精确检查	标准答案、约束、单元测试	可靠、可重复	只适合可验证任务
执行环境	运行代码、执行 SQL、调用模拟器	直接看行为结果	环境成本与覆盖率有限
Reward / Judge Model	另一个模型给分或比较	覆盖开放任务	会有偏差、幻觉、reward hacking
人类	专家审查	语义判断能力强	慢、贵、不可扩展到每次请求

如果任务具有明确、自动化、难作弊的验证信号，例如数学最终答案、代码测试、形式化证明检查，那么 test-time search 往往更容易做得扎实。因为系统不仅能“想”，还能得到较可信的反馈。

科研警告

用 LLM-as-a-Judge 做 verifier 时，必须把 judge 当作一个会出错的测量仪器，而不是绝对真值。

10. Outcome Verifier 与 Process Verifier

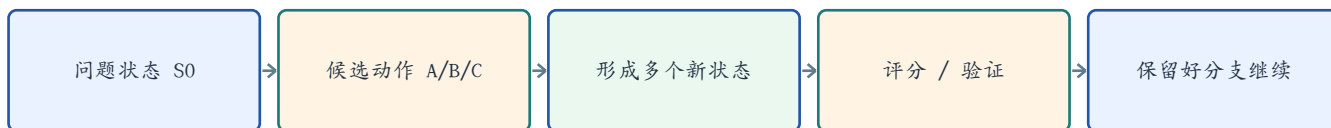
类型	看什么	优点	风险
Outcome / final-answer verifier	只检查最终结果	简单、便宜；对可验证任务很强	无法指出中间哪一步错
Process verifier	逐步检查中间推理步骤	可做早期剪枝、局部纠错	标注/训练难；“看起来合理”仍可能不是因果真实过程

想象树搜索：如果必须等到完整解答生成完才能知道对错，就会浪费很多分支；如果每走一步都能判断“这一步已经不可能成功”，就可以及时剪枝。因此 process-level signal 对搜索效率非常重要。

但是，过程评分也更容易遇到“语言看起来很漂亮，所以被判为好步骤”的问题。对于研究者来说，应该优先寻找真正可执行、可验证的过程信号。

11. Search：从“一条推理链”升级成“搜索空间”

一条 CoT 是一条路径；Search 则允许在某个中间点产生多个分支，并根据评分保留有希望的分支。



树搜索、beam search、Tree-of-Thought 等方法共享的基本结构

这和经典 AI 很像：模型不再只是一次性“生成答案”，而是同时扮演 proposal generator，搜索算法负责管理候选状态，verifier 负责评估。

搜索的成本会很快膨胀。如果每个节点分 4 个分支，搜索深度是 5，那么未经剪枝就可能出现 $4^5=1024$ 条末端路径。因此 verifier 质量和剪枝策略决定了算力是否被有效利用。

12. Tool Use 也是一种“把 test-time compute 外包”

推理并不一定全部由 LLM 内部完成。模型可以把某些子问题交给更可靠的外部计算系统：

- 算术 -> 计算器 / Python
- 事实更新 -> 检索 / 数据库
- 代码正确性 -> 编译器 / 单元测试
- 规划执行 -> 环境反馈 / simulator
- 形式化逻辑 -> theorem prover / constraint solver

这时系统的能力来源不再只是语言模型参数，而是：

LLM proposal + Tool execution + Feedback + Revision

重要视角

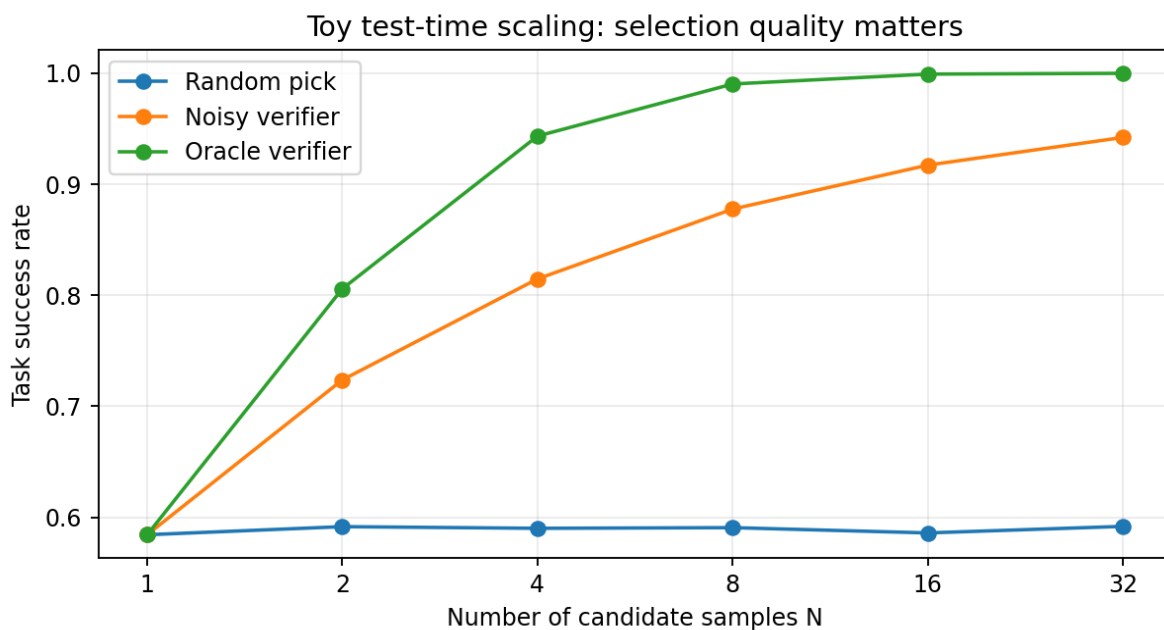
Agent 可以看成一种 test-time computation

architecture: 它允许模型在一次任务中反复“观察 -> 计划 -> 行动 -> 获取反馈 -> 再计算”。

13. 一个数值实验：为什么 N 增大并不等于性能无限增长？

配套代码构造了很多难度不同的任务。对同一道任务的多次采样共享一个 latent difficulty，因此错误天然存在相关性。然后比较三种策略：

- Random pick: 生成 N 个候选，但随便挑一个。
- Oracle best-of- N : 只要有正确候选，就一定能选出来。
- Noisy verifier: 正确候选平均分更高，但评分器并不完美。



图中最重要的不是具体数值，而是三条曲线之间的差距：候选数量只是潜在机会，选择器决定机会能兑现多少。

14. 实验输出怎么读？

下面是本次固定随机种子的实际运行结果：

```
N | random-pick | oracle-best-of-N | noisy-verifier
```

```
--+-----+-----+-----
```

```
1 | 0.5842 | 0.5842 | 0.5842
```

```
2 | 0.5915 | 0.8058 | 0.7237
```

```
4 | 0.5900 | 0.9434 | 0.8147
```

```
8 | 0.5906 | 0.9903 | 0.8776
```

```
16 | 0.5858 | 0.9992 | 0.9172
```

```
32 | 0.5917 | 0.9999 | 0.9422
```

Mean single-sample success $p = 0.5876$

If samples were truly independent with constant p ,

$P(\text{at least one success}) = 1 - (1-p)^N$

Random pick 基本保持在单次采样水平，因为虽然花了更多算力，却没有利用额外候选。Oracle 曲线接近“候选集合里只要出现一次正确就算成功”的上限。Noisy verifier 介于两者之间，这更接近现实系统。

关键原理

Test-time compute 的收益 = 候选多样性 × 验证/聚合能力 × 可用于纠错的反馈质量。任何一项接近 0，额外算力都可能浪费。

15. 为什么会出现收益递减？

即使 `test-time compute` 确实有用，也常见“前几倍算力提升明显，继续加算力提升越来越小”。常见原因包括：

原因	直觉
错误高度相关	同一模型重复采样仍在同一个知识盲区/错误模式里打转
Verifier 饱和	候选更多以后，评分器成为瓶颈；甚至会把好答案排错
任务本身已接近上限	简单题 $N=1$ 已能解决，额外算力没有地方发挥
搜索空间膨胀	更多分支意味着更多噪声和更高选择难度
上下文污染	错误中间步骤和大量候选进入上下文，反而干扰最终决策

所以研究 `test-time scaling` 时，不应该只画“性能 vs N ”，还要同时观察：成本、延迟、token、候选多样性、`verifier accuracy`、错误相关性和任务难度。

16. “多想”为什么有时反而更差？

这是非常重要的反直觉点。Test-time compute 不是单调保证。

- Overthinking: 本来正确的直觉答案被后续错误推导推翻。
- Error reinforcement: 模型在错误前提上生成越来越多自洽解释。
- Judge bias: 更多候选让错误评分器有更多机会选中“看起来更漂亮”的错答案。
- Context dilution: 上下文被大量低质量 reasoning 填满，关键信息反而变得不突出。
- Tool failure: 更多工具调用也意味着更多接口失败、权限错误和脏数据机会。
- Latency / budget exhaustion: 系统可能在价值很低的分支上耗尽预算。

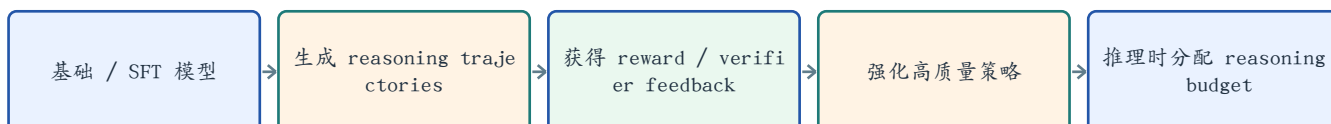
实验含义

任何“增加 reasoning budget”的方法都应该和固定预算/动态预算策略比较，而不是默认“越长越好”。

17. Reasoning Model 不只是“Prompt 里写一句 think step by step”

早期 CoT 研究让大家看到：合适的提示方式可以诱导已有模型产生更长的中间步骤。但现代 reasoning-oriented model 往往还会在训练阶段专门优化“解决复杂任务的轨迹”。

一个抽象流程可以写成：



尤其在数学、代码、形式化任务中，如果最终答案或执行结果可以自动验证，就可以产生相对清晰的 reward。训练过程可能因此学到：什么时候分解、什么时候检查、什么时候回溯、什么时候继续搜索。

不要混淆

训练 reasoning model 是 training-time optimization; 让一个已经训练好的 reasoning model 在某道题上“想更久”是 test-time scaling。二者可以同时存在。

18.

看到“推理文本”，不要自动把它当作模型内部全部真实过程

自然语言 reasoning trace 对用户和研究者很有用，因为它是一个可观察接口。但需要保持科学上的谨慎：模型内部真正进行的是高维向量计算，而输出的文字只是其中一个外显产物。

- 一条解释可能在答案已经倾向确定后才被生成，形成事后合理化。
- 模型可能省略实际重要的隐含计算，也可能输出并未真正决定答案的“漂亮步骤”。
- 对 reasoning trace 的忠实性研究，本身就是一个独立问题。

因此在实验里，最好把 reasoning trace

当作行为数据：可以测量、干预、比较，但不要只凭它就断言内部因果机制。

19. 从 LLM Reasoning 走向 Agent：区别在哪里？

层级	LLM reasoning	Agentic reasoning
状态来源	主要来自当前上下文与内部生成	还可来自环境、memory、工具、文件、数据库
动作空间	主要是生成 token	可生成 token，也可执行 API / 工具 / 环境动作
反馈	下一 token 的上下文、内部 self-check	真实工具结果、环境状态、其他 agent 消息
循环	可以是一条长 reasoning trace	通常显式地 observe-plan-act-feedback 多轮循环
主要风险	幻觉、错误推理、过度思考	再叠加工具错误、权限、长期记忆污染、交互级联

可以把 Agent Runtime 看成一种“把 test-time reasoning 扩展到外部世界”的系统层。LLM 不只在 token 空间里计算，还能通过行动改变下一轮输入。

20. 对 Multi-Agent 科研最重要的一条：Compute-Matched Baseline

假设实验 A：一个 Agent 调用模型 1 次；实验 B：4 个 Agent，各自生成答案、互评两轮、再让一个聚合 Agent 总结。B 的总模型调用和 token 可能是 A 的十几倍。

如果 B 准确率更高，至少有三种解释：

- 真正的多 Agent 协作机制有效；
- 只是因为总采样次数更多，类似 Best-of-N；
- 只是因为总 reasoning token / test-time compute 更多。

所以更严谨的设计应该加入：

Baseline	控制目的
Single Agent, 1 call	最基础成本基线
Single Agent, same total tokens	控制 reasoning token 总预算
Single Agent, Best-of-N	控制候选数量 / sampling compute
Single Agent + same verifier	控制“是否有评分器”这一额外能力
Multi-Agent, same budget	在近似相同总预算下测试协作结构本身

科研判断

如果 multi-agent 只有在多花 10 倍 token 时才赢，而 compute-matched single-agent 不输，那么论文应谨慎地把收益归因于“更多 test-time compute”，而不是直接归因于“集体智能”。

21. Agent Systems 中，怎么记账 test-time compute?

没有一个单一指标能完美衡量所有系统，但至少应该记录下面这些基本量：

资源量	为什么要记录
Model calls	最直接的交互次数；多 Agent 很容易急剧增加
Input tokens	长上下文、历史、memory、agent 消息都会累积
Output / reasoning tokens	模型实际生成了多少计算性文本
Wall-clock latency	并行 Agent 和串行 Agent 的真实用户成本不同
API / compute cost	部署可行性的重要指标
Tool calls	搜索、代码执行、数据库访问也有资源成本
Verifier calls	选择候选所付出的额外评估成本

对科研论文而言，最好同时报告“性能”和“成本”，甚至画成 Pareto frontier：在不同预算下，哪种系统能获得更好的准确率/可靠性。

22. 错误相关性：为什么它是 reasoning / multi-agent scaling 的核心变量？

如果多个候选独立犯错，那么增加候选数量很有价值；如果所有候选都犯同一种错，再多候选也只是重复。可以把它类比为投资组合：资产数量多不等于风险低，关键是相关性。

对 Agent Systems 也一样：

Diversity of trajectories -> Lower error correlation -> More useful aggregation potential

因此未来如果你研究多 Agent，不妨把“平均准确率”之外再加一组系统变量：

- pairwise disagreement / agreement rate;
- error correlation matrix;
- 错误类型重合度;
- 不同模型/经历/角色带来的 candidate diversity;
- 聚合后错误是否被消掉，还是形成 wrong consensus。

科研洞察

从这个视角看，“更多 Agent”本身不是关键；真正可能产生集体增益的是：在预算可控的前提下，系统是否获得了有用的错误去相关性，并能通过通信/验证把它转化为正确决策。

23. 一个合格的 Test-Time Scaling 实验应该怎么设计？

可以使用下面这套最低闭环：

步骤	要做什么
1. 定义任务	选择有清晰评价指标的任务；最好能自动验证
2. 定义预算	N、最大 reasoning tokens、模型调用数、工具调用等
3. 设置 baseline	Direct、CoT、Best-of-N、compute-matched single-agent
4. 控制变量	相同模型、相同温度/采样协议、相同工具权限
5. 多随机种子	避免一次采样带来的偶然性
6. 记录中间量	candidate diversity、verifier accuracy、error correlation
7. 画 scaling curve	性能 vs token/calls/cost，而不是只报一个预算点
8. 做失败分析	哪些题越想越错？哪些错误投票无法消除？

这套结构以后可以直接迁移到你的 Agent Systems 研究。

24. 常见误区集中纠正

误区	更准确的理解
“CoT 越长越聪明”	长度只是成本；关键是额外步骤是否增加有效计算/纠错
“采样 32 次就等于能力提升 32 倍”	候选高度相关且 verifier 不完美，收益通常远低于线性
“多数投票一定更可靠”	如果错误相关，多数可以形成稳定的错误共识
“Verifier 是裁判，所以它就是 ground truth”	Verifier 也是模型/程序，也有测量误差与偏差
“多 Agent 赢了，所以协作有效”	先排除 token、calls、Best-of-N、judge 等额外预算
“Reasoning trace 就是模型内部真实思维”	它是可观察行为，但不应自动等同完整因果机制
“推理模型只是在 prompt 里加 think step by step”	现代系统常结合专门训练、reward/verifier 与推理阶段预算

25. 本讲核心术语表

术语	中文直觉	你要记住的最小含义
Reasoning	推理	用中间计算状态解决复杂任务
Chain-of-Thought (CoT)	思维链	显式生成中间自然语言步骤
Test-Time Compute	推理时计算	对当前任务额外投入的计算预算
Best-of-N	N 选 1	生成多个候选，再选择一个
Self-Consistency	自治投票	多条推理轨迹最终答案聚合
Verifier	验证器	判断候选质量/正确性的组件
Outcome Reward	结果奖励	根据最终结果评分
Process Reward	过程奖励	对中间步骤提供评分
Search	搜索	在多条候选路径之间扩展、剪枝和选择
Compute-matched baseline	等算力基线	用相近总预算比较不同系统结构
Error Correlation	错误相关性	多个候选是否倾向犯同一种错

26. 配套实验：你应该观察什么？

运行：

```
python 04_test_time_compute_demo.py
```

不要只盯着最高成功率，而要按下面顺序观察：

- Random pick 随 N 增大是否明显变强？为什么不会？
- Oracle best-of- N 增长有多快？它代表什么上限？
- Noisy verifier 与 oracle 的差距说明什么？
- 为什么代码故意让每个 task 有自己的 p_{task} ，而不是所有采样都共享一个固定 p ？
- 把 verifier 的“正确候选加分”从 1.2 改成 0.4 / 2.0，会发生什么？
- 把 Beta 分布换成更极端的任务难度分布，scaling curve 会怎么变？

建议

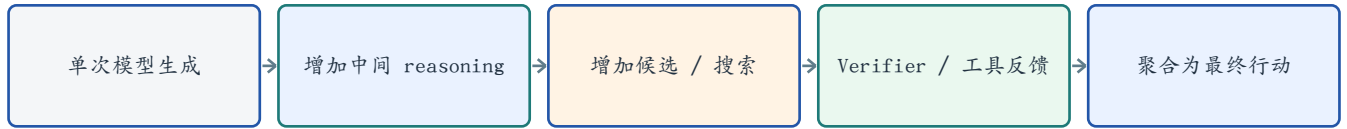
真正做科研时，把这个玩具实验替换成真实 benchmark； N 换成实际模型调用次数；success 换成任务正确率；verifier 换成 judge、测试环境或规则。

27. 练习题

先独立回答，再看答案文件。

题号	题目
1	用自己的话解释 Training Compute 与 Test-Time Compute 的区别。
2	为什么生成 16 个候选后随便选一个，通常不会因为 N=16 就明显提升准确率？
3	Best-of-N 的收益依赖哪两个关键条件？
4	为什么错误相关性会破坏 Self-Consistency / 多数投票？
5	Outcome Verifier 和 Process Verifier 有什么区别？
6	为什么 LLM-as-a-Judge 不能直接当 ground truth？
7	“多 Agent 比单 Agent 高 8%” 为什么还不能直接说明协作有效？
8	设计一个 compute-matched baseline，比较 4-Agent 系统和单 Agent。
9	举一个“多想反而更差”的机制。
10	为什么 Tool Use 可以被看成一种 test-time computation？

28. 本讲总结：形成一张完整心智图



Test-time compute 的关键不是“更长”，而是让额外计算进入有效的探索 - 验证 - 纠错闭环

最后只保留 6 条最重要的结论：

- 1. Reasoning 可以先理解为“为解决当前任务增加中间计算状态”，不必先诉诸神秘解释。
- 2. CoT 的价值来自分解、状态维护、搜索和验证机会，而不是文字长度本身。
- 3. Test-time compute 和训练算力是不同 scaling 轴：前者针对每个具体请求。
- 4. 更多候选只有在有多多样性、可靠验证和有效聚合时才能变成更高性能。
- 5. 错误相关性与 verifier 质量决定 scaling 上限；更多计算也可能越想越错。
- 6. Agent / Multi-Agent 研究必须控制总 token、调用数和验证成本，否则容易把“多算力”误当成“协作机制”。

下一讲

第19讲：Inference - 你按下“发送”以后，模型内部到底发生什么？将把 tokenizer、prefill、Transformer、logits、sampling、decode loop 串成完整在线推理流程。